

Nutzerhandbuch

Inhalt

1. Was ist diese Anwendung?
2. Projektstruktur
3. Installation & Start
4. Die Tabs im Detail (mit Screenshots)
 - 4.1 Vogelgenerierung
 - 4.2 Kameras
 - 4.3 Übersicht (Triangulations-Pipeline)
 - 4.4 Trajektorie berechnen & Feldlerchen-Klassifikation
 - 4.5 2D-Kamerabild
5. Datenfluss (Gesamtüberblick)
6. JSON-Datenformate
7. Typischer Arbeitsablauf
8. Bekannte Einschränkungen & offene Punkte
9. Wo man im Code weiterarbeitet
10. Kontakt & Übergabe-Hinweise

1. Was ist diese Anwendung?

Ein lokales Dash/Plotly-Webinterface (**webinterface.py**), das den gesamten Simulationszyklus für das Vogel-Kamera-Triangulationssystem abbildet:

- Künstliche Vogelflugbahnen erzeugen, inklusive eines Sonderfalls für Feldlerchen mit einem eigenen, klar erkennbaren Flugmuster.
- Ein virtuelles Kamera-Setup (mehrere Raspberry Pis) definieren.
- Prüfen, welche Vogelpositionen von welcher Kamera „gesehen“ würden (Sichtvolumen-Filterung).
- Aus den 2D-Kamerabildern mehrerer Kameras die reale 3D-Position zurückrechnen (Triangulation).
- Aus Einzelbeobachtungen zusammenhängende Flugbahnen (Tracks) bilden.
- Jeden Track anhand seines Flugmusters als „Feldlerche“ oder „kein Treffer“ klassifizieren — direkt in der Web-Oberfläche.

Dieses Handbuch verbindet die technische Referenz (Code-Architektur, Datenformate, Klassifikator-Logik) mit Screenshots, die zeigen, wie jeder Schritt in der Oberfläche tatsächlich aussieht.

2. Projektstruktur

2.1 Dateiliste (helper_functions/)

- **pi_view_simulation.py** — Geometrie: Sichtvolumen, Projektion, Kameraebene
- **bird_generation.py** — Erzeugt simulierte Vogel-Flugbahnen (Standard-Vögel + Aufruf des Feldlerchen-Generators)
- **bird_generation_layout.py** — Dash-Layout für den Tab „Vogelgenerierung“
- **skylark_generation.py** — Erzeugt das spezielle Feldlerchen-Singflug-Muster (Datengenerierung)
- **skylark_classifier.py** — FeatureExtractor + Classifier, der Tracks anhand ihres Flugmusters als „Feldlerche“ einstuft. Vollständig in webinterface.py eingebunden (Tab „Trajektorie berechnen“).
- **detected_positions.py** — Berechnet/speichert erkannte (triangulierte) Positionen
- **generate_plots.py** — Erstellt 2D-/3D-Plotly-Figuren, Statistiken
- **helper_ui.py** — Wiederverwendbare Dash-UI-Bausteine (Inputs, Slider ...)
- **additional_styles.py** — Zentrale Style-Konstanten (Status-Farben, Feld-Styles)

Namenshinweis: webinterface.py importiert das Modul als `helper_functions.skylark_classifier`. Der Docstring am Kopf der Datei selbst bezeichnet sie intern als „feldlerche_classifier.py (v3)“ — derselbe Code, nur mit abweichendem internen Namen im Kommentar.

2.2 Ordnerbaum

```

projekt/
+-- webinterface.py          # Haupt-App: Dash-Server, Layout, alle Callbacks,
|                             # Klassifikator-Integration
+-- trajectory.py           # Eigenstaendiges Skript: bildet zusammenhaengende
|                             # Tracks aus Einzelbeobachtungen (Greedy-Zuordnung)
+-- helper_functions/
|   +-- pi_view_simulation.py # Geometrie: Sichtvolumen, Projektion, Kameraebene
|   +-- bird_generation.py   # Generator: normale Vogel-Flugbahnen (ruft
|   |                         # skylark_generation.py fuer Feldlerchen mit auf)
|   +-- bird_generation_layout.py # Dash-Layout fuer Tab "Vogelgenerierung"
|   |                         # (Parameterformular, 2D-/3D-Vorschau)
|   +-- skylark_generation.py # Generator: erzeugt das Feldlerchen-Singflug-
|   |                         # Muster (Steigen -> Kreisen -> Sinken) als Testdaten
|   +-- skylark_classifier.py # Classifier: FeatureExtractor + Regelwerk, das
|   |                         # Tracks als Feldlerche einstuft (Tab "Trajektorie
|   |                         # berechnen")
|   +-- detected_positions.py # Berechnet/speichert triangulierte (erkannte)
|   |                         # 3D-Positionen aus den Kamera-Projektionen
|   +-- generate_plots.py    # Erstellt 2D-/3D-Plotly-Figuren und die
|   |                         # Statistik-Kacheln (Gesamtpunkte, Hoehe, etc.)
|   +-- helper_ui.py         # Wiederverwendbare Dash-UI-Bausteine: Inputs,
|   |                         # Slider, Datumsfelder, Statistik-Karten
|   +-- additional_styles.py # Zentrale Style-Konstanten: Status-Farben
|   |                         # (Erfolg/Fehler), Feld- und Layout-Styles
+-- vogel_flugbahnen.json    # Generierte Rohdaten (Output "Vogelgenerierung")
+-- vogel_flugbahnen_determined.json # Ergebnis von trajectory.py (Tracks) - Input
|                             # fuer den Klassifikator
+-- vogel_bilder.json        # Projizierte 2D-Kamerabilder je Pi
+-- vogel_position_erkannt.json # Triangulierte/"erkannte" 3D-Positionen

```

3. Installation & Start

Voraussetzungen: Python 3.10+, Pakete: dash, pandas, plotly.

```

pip install dash pandas plotly
python webinterface.py

```

Die App läuft unter `http://localhost:8060` (Port über `APP_PORT` änderbar). Debug-Modus ist standardmäßig aktiv.

4. Die Tabs im Detail

4.1 Tab „Vogelgenerierung“

Erzeugt synthetische Flugbahnen und schreibt sie nach `vogel_flugbahnen.json`.

Parameter	Bedeutung
Anzahl Vögel (1-500)	Anzahl simulierter „normaler“ Vögel
Anzahl Feldlerchen (0-500)	Zusätzliche Vögel mit speziellem Singflug-Muster
Messabstand [s] (1-60)	Zeitintervall zwischen zwei Positionsmessungen
Simulationsdatum	Startdatum für zufällige Startzeiten
Zufalls-Seed	Leer = zufällig; Text/Zahl = reproduzierbar
X/Y/Z Min/Max	Raumgrenzen
Initiale Streuung	Zufälliger Versatz der Startposition
Min/Max Geschwindigkeit	Geschwindigkeitsbereich (nur normale Vögel)
Positionsrauschen	Simuliertes Messrauschen
Varianzparameter	Richtungs-/Geschwindigkeits-/Höhenänderung, Steiggeschwindigkeit

Ablauf: Parameter setzen -> „JSON generieren“ -> Status, Statistiken, 2D-Karte und 3D-Plot erscheinen.
Optional „JSON herunterladen“.

Flugtrajektorien

Berechnung und 3D-Visualisierung

Dashboard / Plotly

Vogelgenerierung

Kameras

Übersicht

Trajektorie berechnen

2D-Kamerabild

PARAMETER

Simulation

Allgemeine Einstellungen

Anzahl Vögel

50

Anzahl Feldlerchen

1

Messabstand [Sekunden]

5

Simulationsdatum

08.07.2026

Zufalls-Seed (leer = zufällig)

Raumgrenzen

X Min

0

X Max

600

Y Min

0

Y Max

600

Z Min

50

Z Max

300

Initiale Streuung (Offset)

0

500

1000

1500

2000

1000

Zufälliger Offset bei der Startposition, damit Vögel nicht exakt in der Mitte starten.

Geschwindigkeit

Min Geschwindigkeit

4

Max Geschwindigkeit

16

Fehlerraten

Positionsrauschen

0

0.1

0.2

0.3

0.4

0.5

0.6

0.7

0.8

0.9

1

0.08

Varianz der Flugbahnen

Richtungsänderung

0

0.05

0.1

0.15

0.2

0.25

0.3

0.04

Geschwindigkeitsänderung

0

0.1

0.2

0.3

0.4

0.5

0.6

0.7

0.8

0.9

1

0.03

Vertikale Bewegung

0

0.1

0.2

0.3

0.4

0.5

0.6

0.7

0.8

0.9

1

0.05

Steiggeschwindigkeit ändern

0

0.015

0.03

0.045

0.06

0.075

0.09

0.1

0.01

JSON generieren

GENERIERTE DATEN

voget_flugbahnen.json

Status: Parameter einstellen und auf „JSON generieren“ klicken.

Nach der Generierung werden hier die Statistiken angezeigt.

JSON herunterladen

Anschließend kann im Tab „Übersicht“ die vorhandene trajectory.py ausgeführt werden.

2D-KARTE DER FLUGBAHNEN

Nach der Generierung erscheint hier die 2D-Karte.

3D-PLOT DER FLUGBAHNEN

Nach der Generierung erscheint hier der 3D-Plot.

Abb. 1: Parameter-Formular vor der Generierung — 2D-Karte und 3D-Plot sind noch leer.

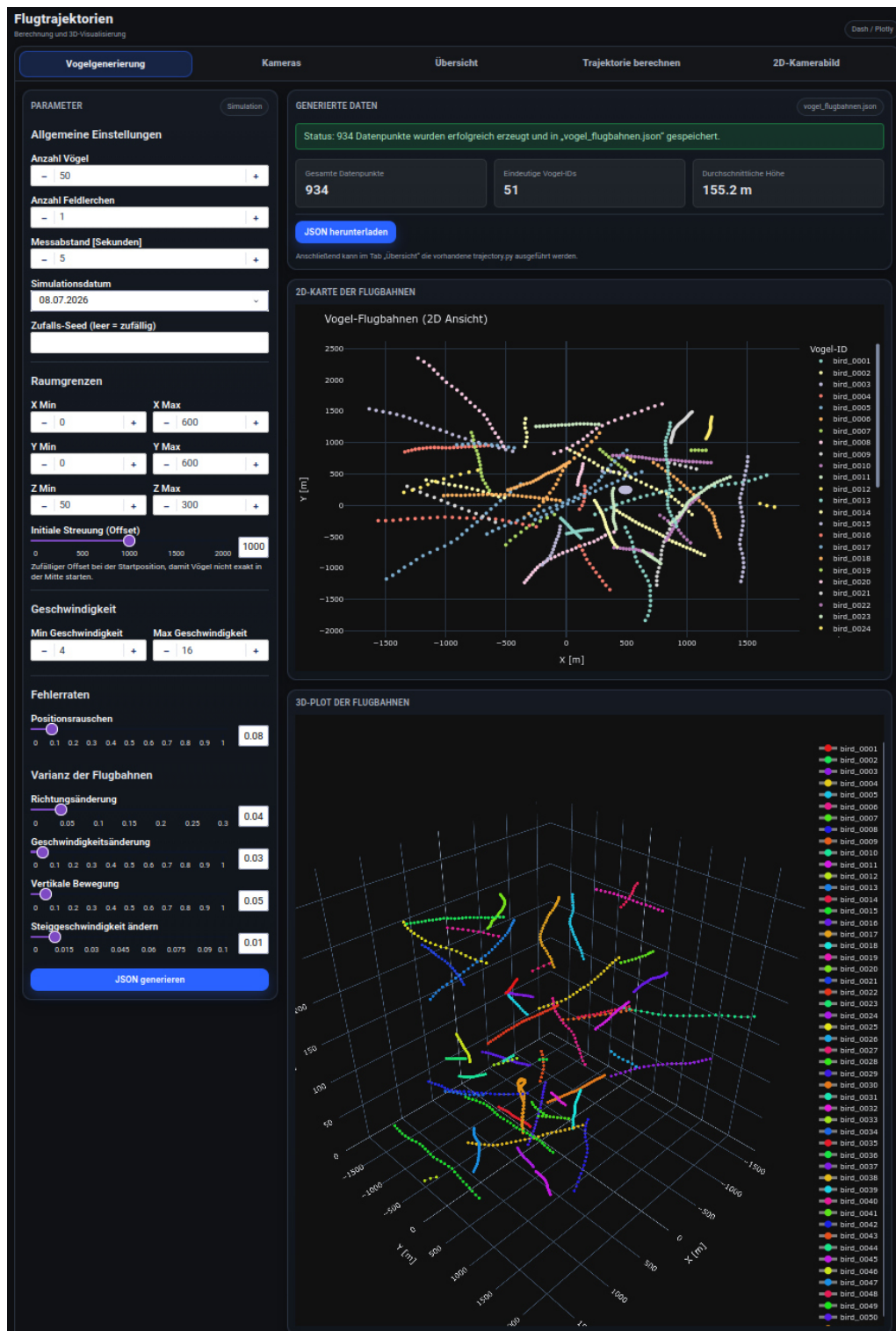


Abb. 2: Ergebnis nach „JSON generieren“ — 934 Datenpunkte, Statistik-Kacheln, 2D-Karte und 3D-Plot der Flugbahnen, farblich nach Vogel-ID.

4.2 Tab „Kameras“

Editierbare Tabelle mit dem Kamera-Setup: standardmäßig drei Raspberry Pis (Ursprung, X-Achse, Y-Achse) mit Position (X/Y/Z) und Blickrichtung (Yaw/Pitch/Roll). Werte können direkt in der Tabelle geändert werden und wirken sich live auf die 3D-Ansicht in „Übersicht“ und „Trajektorie berechnen“ aus. Die Sichtweite (view_length) ist im Code hart auf 400.0 gesetzt.



Name	X	Y	Z	Yaw um Z [°]	Pitch hoch/runter [°]	Roll [°]
Raspberry Pi Ursprung	0	0	0	40	45	0
Raspberry Pi X-Achse	300	0	0	150	45	0
Raspberry Pi Y-Achse	0	300	0	300	45	0

Abb. 3: Kamera-Setup-Tabelle mit den drei Standard-Pis.

4.3 Tab „Übersicht“ — die zentrale Triangulations-Pipeline

Drei Buttons, die nacheinander benutzt werden:

- **„Vogel-JSON laden“** — liest `vogel_flugbahnen.json` und zeigt alle generierten Vogelpositionen (ungefiltert) in der 3D-Ansicht.
- **„Punkte in Pyramiden filtern“** — prüft für jede Kamera, welche Punkte innerhalb ihres pyramidenförmigen Sichtvolumens liegen, behält nur die sichtbaren Punkte, **überschreibt `vogel_flugbahnen.json` unwiderruflich** und speichert die Projektionen in `vogel_bilder.json`.
- **„Position bestimmen“** — berechnet aus `vogel_bilder.json` über Triangulation die geschätzten 3D-Positionen/-Vektoren und speichert sie in `vogel_position_erkannt.json`.

Farbcodierung in der 3D-Grafik: rote Diamanten = Kameras, orange Linien/Flächen = Blickrichtung und Sichtebeine, grüne Linien = Kamera->Projektionspunkt, farbige Punkte = Vogelpositionen.

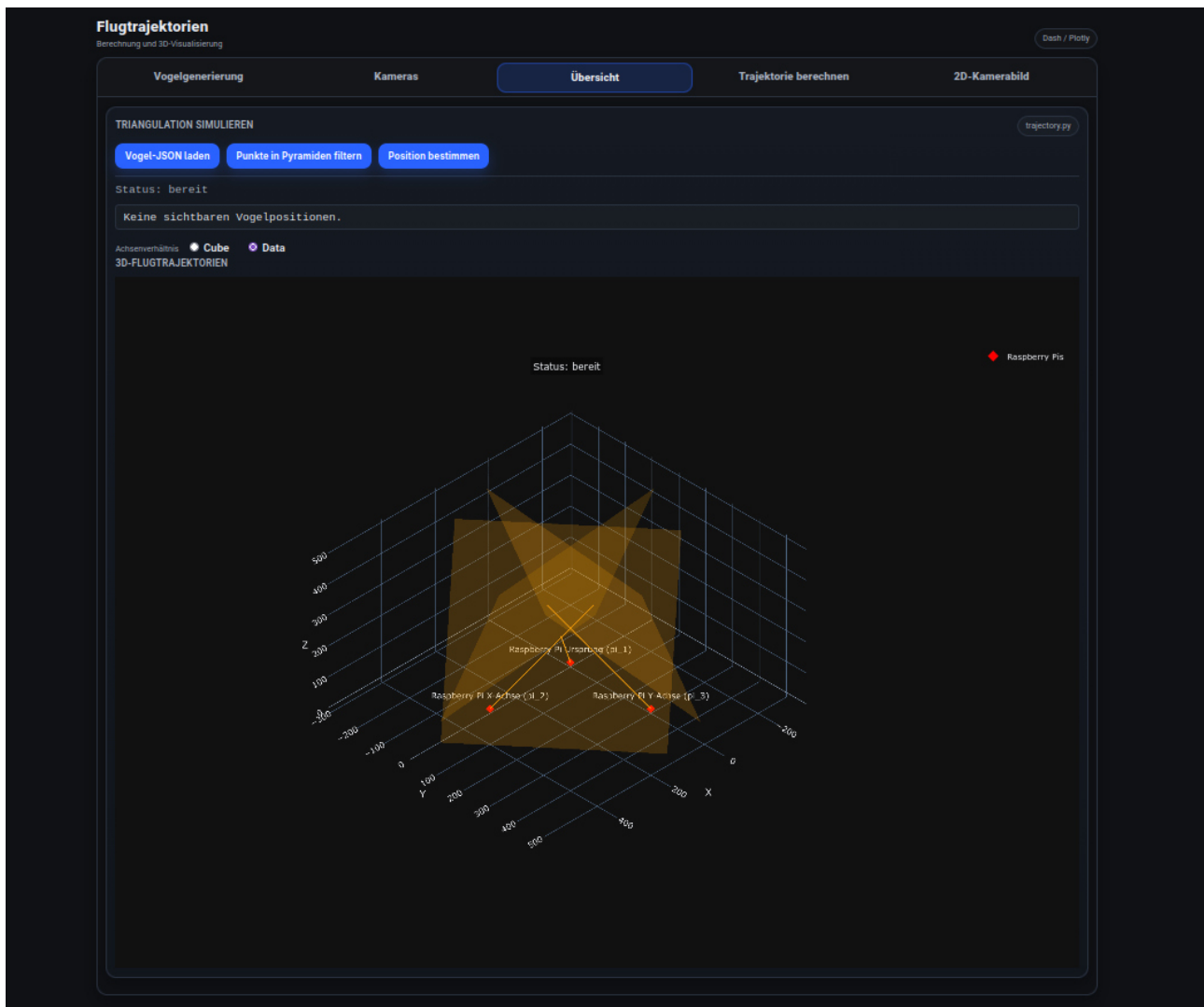


Abb. 4: Ausgangszustand — nur Kameras (rote Diamanten) und Sichtvolumen (orange Pyramiden) sichtbar, Status „bereit“.

Flugtrajektorien

Berechnung und 3D-Visualisierung

Dash / Plotly

Vogelgenerierung

Kameras

Übersicht

Trajektorie berechnen

2D-Kamerabild

TRIANGULATION SIMULIEREN

trajectory.py

Vogel-JSON ladenPunkte in Pyramiden filternPosition bestimmen

Status: vogel_flugbahnen.json geladen. Bitte anschließend „Punkte in Pyramiden filtern“ klicken.

```
bird_0001: 10 Position(en) - (518.13, 33.27, 213.38); (487.85, 38.02, 213.38); (450.81, 67.48, 213.38); (420.17, 83.85, 213.17); (393.46, 181.88, 214.97) ...
bird_0002: 19 Position(en) - (-420.30, 1893.87, 99.22); (-483.88, 1148.75, 98.97); (-378.81, 1201.23, 98.73); (-350.85, 1233.71, 98.39); (-327.98, 1307.83, 98.07) ...
bird_0003: 9 Position(en) - (-380.30, -418.32, 73.63); (-437.08, -418.81, 73.74); (-414.31, -418.48, 73.88); (-371.78, -428.82, 73.88); (-329.47, -427.14, 73.98) ...
bird_0004: 11 Position(en) - (-288.73, 734.33, 91.32); (-211.30, 731.43, 91.41); (-182.81, 749.60, 91.80); (-133.64, 734.87, 90.89); (-83.38, 762.13, 90.68) ...
bird_0005: 23 Position(en) - (-710.88, 758.84, 118.49); (-872.01, 739.03, 118.88); (-829.48, 733.83, 119.18); (-887.12, 711.82, 119.41); (-943.12, 694.36, 119.78) ...
bird_0006: 23 Position(en) - (-442.88, 1288.34, 104.78); (-481.91, 1103.11, 104.90); (-518.42, 1127.21, 105.31); (-537.49, 1092.49, 105.87); (-597.33, 1059.88, 105.98) ...
bird_0007: 8 Position(en) - (-483.93, -385.16, 296.43); (-452.03, -350.22, 296.41); (-381.38, -343.27, 296.48); (-349.38, -329.98, 296.23); (-398.11, -318.37, 295.93) ...
bird_0008: 22 Position(en) - (-1118.77, -830.48, 282.03); (-1089.37, -591.30, 282.33); (-1082.18, -545.82, 282.09); (-1037.10, -497.21, 282.08); (-1018.85, -450.33, 283.83) ...
bird_0009: 8 Position(en) - (-375.21, -323.43, 281.12); (-380.30, -339.81, 288.97); (-482.33, -398.73, 288.87); (-419.34, -621.91, 281.83); (-437.82, -652.18, 281.43) ...
bird_0010: 8 Position(en) - (-784.78, -341.38, 112.48); (-885.82, -282.88, 112.11); (-829.34, -220.33, 112.88); (-853.88, -189.80, 111.76); (-879.34, -114.21, 111.61) ...
bird_0011: 16 Position(en) - (-323.88, 27.48, 189.48); (-282.82, 31.43, 189.29); (-198.79, 78.33, 189.87); (-133.61, 184.83, 189.84); (-73.73, 137.49, 188.98) ...
bird_0012: 9 Position(en) - (-610.19, 1817.44, 188.28); (-688.40, 1885.17, 188.63); (-599.29, 1112.40, 187.88); (-598.82, 1139.73, 187.88); (-579.19, 1288.96, 187.81) ...
bird_0013: 3 Position(en) - (-328.73, 738.92, 288.88); (-332.84, 888.98, 288.39); (-348.79, 582.37, 288.74); (-334.78, 584.26, 288.64); (-378.84, 428.88, 288.32) ...
bird_0014: 7 Position(en) - (-637.88, 96.43, 72.11); (-673.33, 188.34, 72.18); (-1069.31, 118.78, 72.87); (-1543.89, 127.37, 72.42); (-1581.81, 133.19, 72.71) ...
bird_0015: 12 Position(en) - (-103.32, -449.03, 81.88); (-142.38, -310.23, 88.31); (-119.23, -382.29, 88.19); (-93.97, -848.33, 59.88); (-88.88, -718.88, 59.21) ...
bird_0016: 20 Position(en) - (-382.94, 377.03, 298.88); (-289.34, 443.18, 298.33); (-279.69, 388.98, 298.74); (-273.93, 373.48, 297.87); (-269.48, 641.76, 297.39) ...
bird_0017: 24 Position(en) - (-349.18, -38.33, 223.88); (-328.20, -47.97, 224.74); (-491.18, -85.33, 224.89); (-488.48, -88.82, 224.71); (-438.89, -90.39, 224.81) ...
bird_0018: 3 Position(en) - (-340.14, 188.71, 181.28); (-343.37, 138.83, 181.38); (-344.24, 79.31, 181.88); (-348.88, 28.83, 181.88); (-333.81, -22.44, 182.27) ...
bird_0019: 23 Position(en) - (-34.13, 282.93, 85.94); (-28.11, 148.73, 88.21); (-73.12, 93.83, 88.43); (-138.18, 43.12, 88.72); (-192.82, -8.82, 88.79) ...
bird_0020: 16 Position(en) - (-494.78, 329.34, 147.82); (-323.23, 349.89, 148.88); (-337.84, 368.92, 148.87); (-388.31, 383.39, 148.11); (-618.47, 685.28, 148.27) ...
bird_0021: 16 Position(en) - (-281.29, 782.84, 191.84); (-288.24, 628.41, 198.98); (-292.83, 334.88, 198.98); (-382.88, 479.71, 198.93); (-313.83, 485.81, 191.88) ...
bird_0022: 23 Position(en) - (-714.93, 884.88, 188.33); (-782.33, 831.88, 188.27); (-883.73, 818.11, 188.87); (-848.93, 783.88, 187.88); (-883.39, 724.38, 187.81) ...
bird_0023: 7 Position(en) - (-369.77, 334.92, 171.27); (-412.91, 394.34, 171.18); (-437.71, 432.87, 171.13); (-384.33, 589.29, 171.18); (-348.31, 589.83, 171.88) ...
bird_0024: 23 Position(en) - (-1188.37, 488.89, 73.81); (-1088.28, 338.13, 72.94); (-1887.71, 311.39, 72.73); (-1847.31, 285.36, 72.22); (-1828.34, 219.82, 71.91) ...
bird_0025: 28 Position(en) - (-41.93, -338.92, 184.27); (-71.37, -319.83, 184.33); (-98.81, -297.81, 185.83); (-128.85, -277.13, 185.84); (-134.38, -258.82, 188.18) ...
bird_0026: 11 Position(en) - (-183.34, -338.23, 78.93); (-223.82, -387.29, 77.18); (-282.47, -398.32, 77.83); (-294.49, -428.79, 77.23); (-328.38, -448.83, 77.12) ...
bird_0027: 16 Position(en) - (-724.34, 98.83, 287.48); (-787.27, 78.97, 287.83); (-891.29, 85.88, 287.87); (-874.39, 32.83, 287.37); (-858.92, 42.81, 287.88) ...
bird_0028: 23 Position(en) - (-35.83, 173.98, 132.98); (-71.38, 133.13, 133.84); (-87.27, 138.28, 132.93); (-185.97, 189.14, 133.83); (-124.78, 88.28, 133.11) ...
bird_0029: 28 Position(en) - (-842.83, -834.74, 234.14); (-383.79, -878.83, 234.38); (-529.98, -788.38, 234.39); (-478.43, -748.32, 234.48); (-428.28, -791.87, 234.47) ...
bird_0030: 18 Position(en) - (-119.82, -445.16, 74.83); (-181.18, -482.78, 74.93); (-283.88, -477.28, 74.88); (-248.33, -484.84, 74.91); (-293.37, -491.82, 74.74) ...
bird_0031: 12 Position(en) - (-94.81, 881.98, 118.31); (-48.91, 927.94, 118.78); (-1.43, 972.87, 118.91); (-47.12, 1813.83, 119.11); (-98.78, 1882.71, 118.98) ...
bird_0032: 11 Position(en) - (-114.21, 991.88, 281.39); (-98.13, 1828.43, 281.43); (-71.39, 1888.84, 281.28); (-31.23, 1187.78, 281.34); (-33.37, 1148.33, 281.38) ...
bird_0033: 9 Position(en) - (-229.14, 379.78, 144.78); (-293.89, 341.21, 144.88); (-358.88, 388.78, 144.78); (-423.89, 473.28, 144.78); (-497.39, 433.38, 144.88) ...
bird_0034: 21 Position(en) - (-173.81, 438.24, 31.78); (-199.98, 433.33, 31.39); (-228.39, 428.91, 31.38); (-233.27, 428.41, 31.11); (-288.22, 413.84, 31.88) ...
bird_0035: 24 Position(en) - (-629.73, 388.28, 133.37); (-888.31, 442.12, 133.39); (-594.33, 318.32, 133.83); (-588.82, 393.93, 133.87); (-379.17, 873.31, 133.38) ...
bird_0036: 4 Position(en) - (-1841.18, 849.73, 73.38); (-1828.38, 791.87, 73.23); (-999.42, 732.33, 73.13); (-988.94, 873.87, 74.98) ...
bird_0037: 9 Position(en) - (-1889.32, 1831.89, 214.72); (-1819.71, 1838.81, 213.31); (-949.89, 1883.28, 218.84); (-879.88, 1189.83, 218.83); (-889.83, 1133.82, 217.28) ...
bird_0038: 9 Position(en) - (-138.71, 711.12, 88.39); (-128.84, 798.83, 88.98); (-128.94, 888.82, 87.28); (-188.79, 947.82, 87.73); (-88.84, 1824.37, 88.23) ...
bird_0039: 18 Position(en) - (-191.33, 489.81, 188.87); (-143.21, 423.34, 179.89); (-98.28, 444.13, 179.77); (-31.21, 488.73, 179.87); (-7.47, 491.88, 179.47) ...
bird_0040: 13 Position(en) - (-1144.78, -384.77, 184.88); (-1188.17, -383.88, 184.93); (-1213.83, -888.38, 185.13); (-1232.19, -823.87, 184.92); (-1289.81, -842.34, 184.93) ...
bird_0041: 19 Position(en) - (-91.88, -887.19, 134.38); (-35.18, -883.44, 133.97); (-19.32, -877.28, 133.32); (-18.84, -873.43, 133.88); (-32.33, -878.87, 132.38) ...
bird_0042: 18 Position(en) - (-378.78, 488.88, 91.38); (-827.22, 328.21, 91.23); (-888.22, 388.88, 91.83); (-733.82, 883.47, 98.98); (-788.77, 844.83, 98.88) ...
bird_0043: 13 Position(en) - (-398.21, 733.89, 32.32); (-373.24, 882.39, 32.33); (-338.83, 829.91, 32.84); (-343.87, 373.88, 31.27); (-331.98, 322.82, 31.88) ...
bird_0044: 23 Position(en) - (-18.34, -973.88, 87.49); (-32.34, -998.82, 87.33); (-47.89, -1888.83, 87.11); (-82.91, -1819.82, 88.81); (-78.88, -1834.19, 88.18) ...
bird_0045: 9 Position(en) - (-218.14, 183.92, 287.48); (-189.87, 117.82, 287.88); (-122.13, 133.39, 288.28); (-78.88, 137.41, 288.79); (-29.98, 179.49, 289.89) ...
bird_0046: 11 Position(en) - (-874.84, -288.38, 284.82); (-889.17, -241.32, 284.71); (-781.27, -278.18, 284.73); (-711.34, -318.92, 284.88); (-719.37, -348.38, 284.38) ...
bird_0047: 28 Position(en) - (-338.83, -847.77, 224.88); (-273.84, -881.88, 224.38); (-219.84, -881.88, 223.88); (-184.79, -784.38, 223.21); (-113.43, -733.43, 222.48) ...
bird_0048: 12 Position(en) - (-298.79, -877.18, 193.43); (-229.48, -878.87, 193.21); (-187.88, -887.24, 192.83); (-188.88, -837.38, 192.74); (-47.28, -839.79, 192.19) ...
bird_0049: 8 Position(en) - (-44.22, 377.18, 184.73); (-52.83, 334.89, 184.83); (-83.17, 333.37, 184.78); (-74.92, 311.99, 184.88); (-88.13, 298.41, 184.48) ...
bird_0050: 21 Position(en) - (-23.78, -482.21, 249.28); (-81.39, -324.33, 249.87); (-98.82, -388.29, 249.37); (-148.98, -842.32, 249.39); (-188.97, -893.34, 249.34) ...
lark_0001: 83 Position(en) - (-437.83, 182.88, 31.82); (-437.88, 182.31, 32.38); (-438.13, 181.84, 34.84); (-448.38, 181.44, 38.83); (-443.33, 182.98, 82.84) ...
```

Abb. 5: Nach „Vogel-JSON laden“ — alle Vogelpositionen als blaue Punkte (ungefiltert), Positions-Liste links.

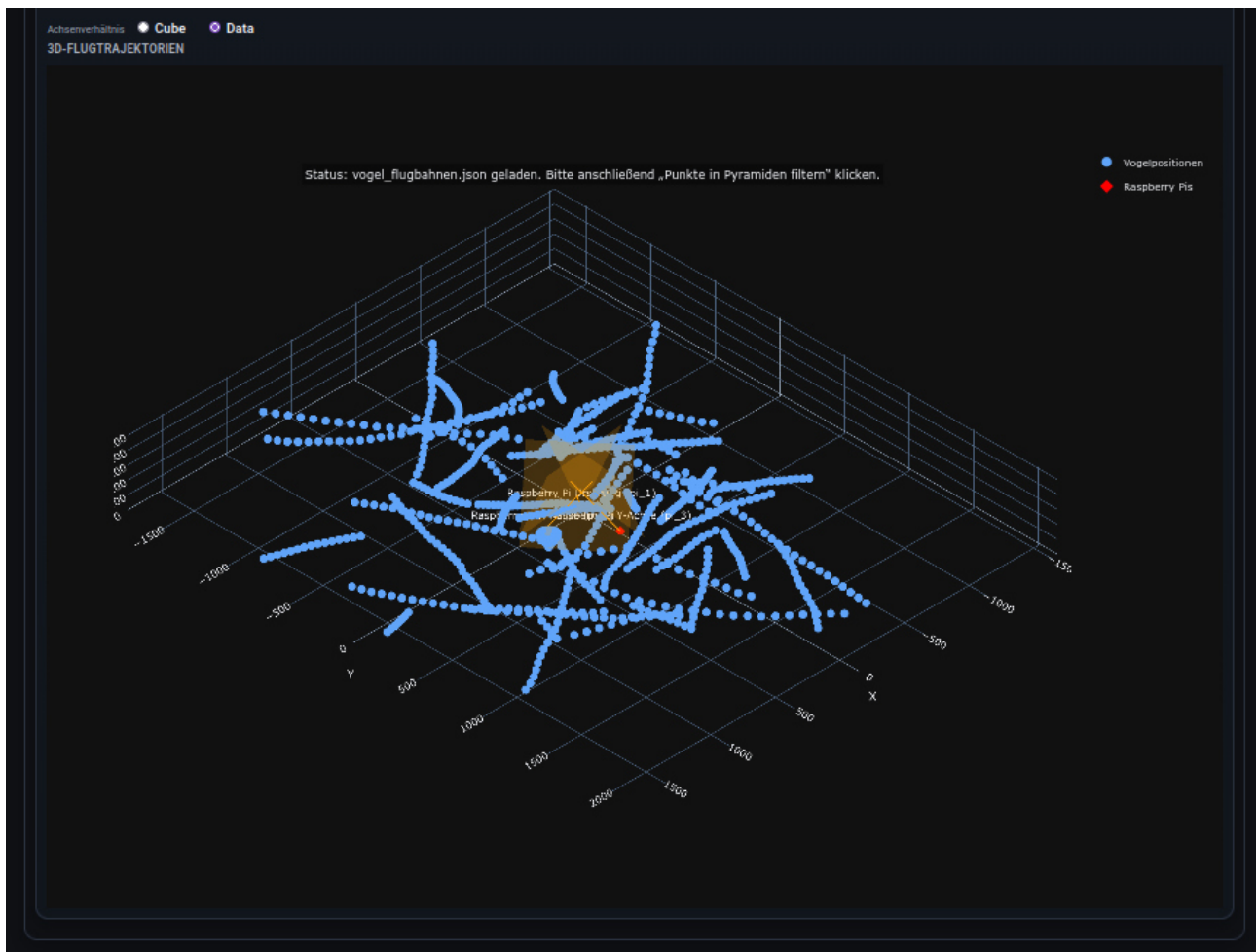


Abb. 6: Dieselbe Ansicht, Achsenverhältnis „Cube“, näher gezoomt — die blauen Flugbahnen liegen erkennbar zum Großteil außerhalb der orangen Sichtpyramiden.

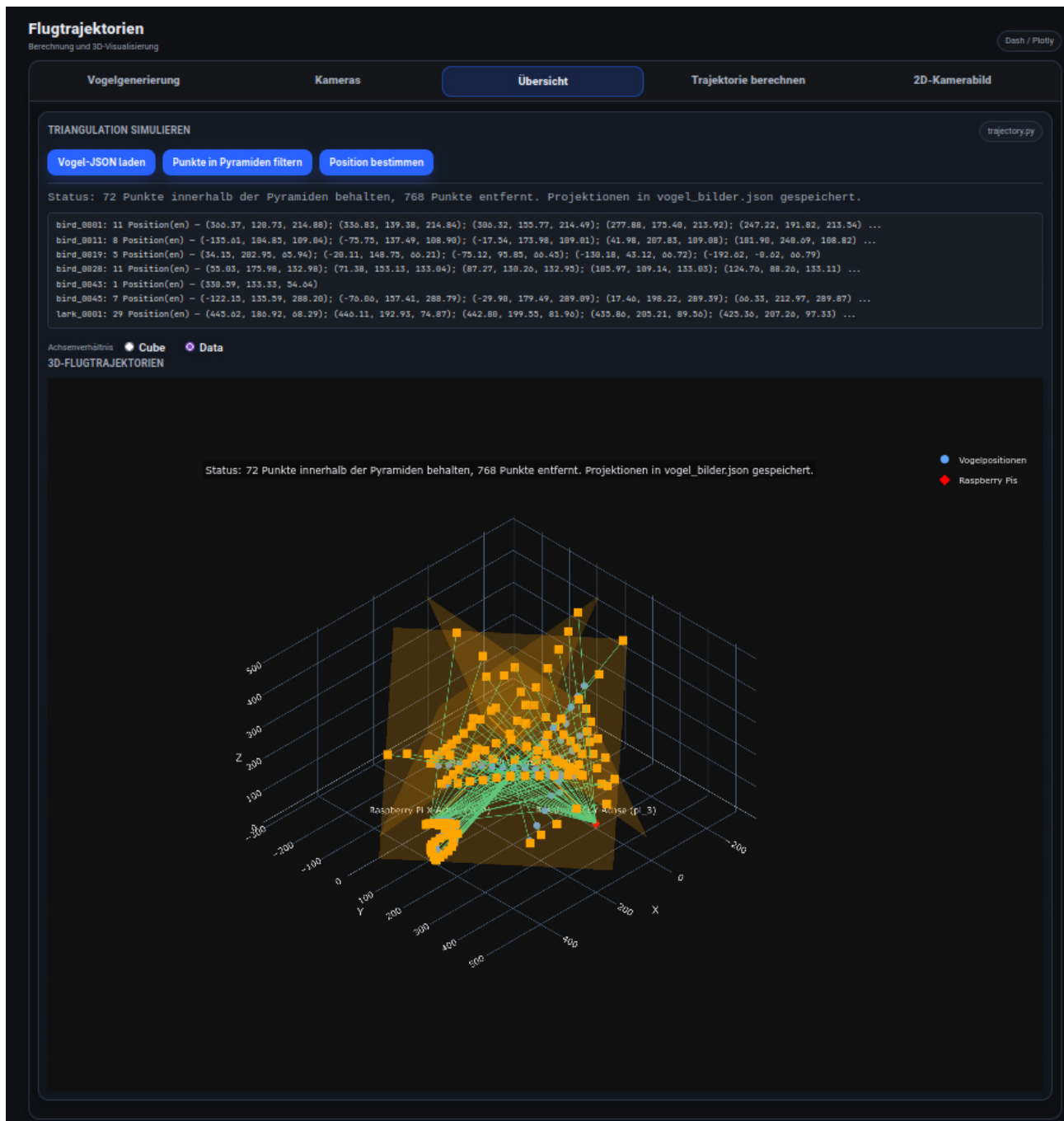


Abb. 7: Nach „Punkte in Pyramiden filtern“ — 72 von 840 Punkten behalten (orange Quadrate), grüne Projektionslinien zu den Kameras; 768 Punkte wurden entfernt.

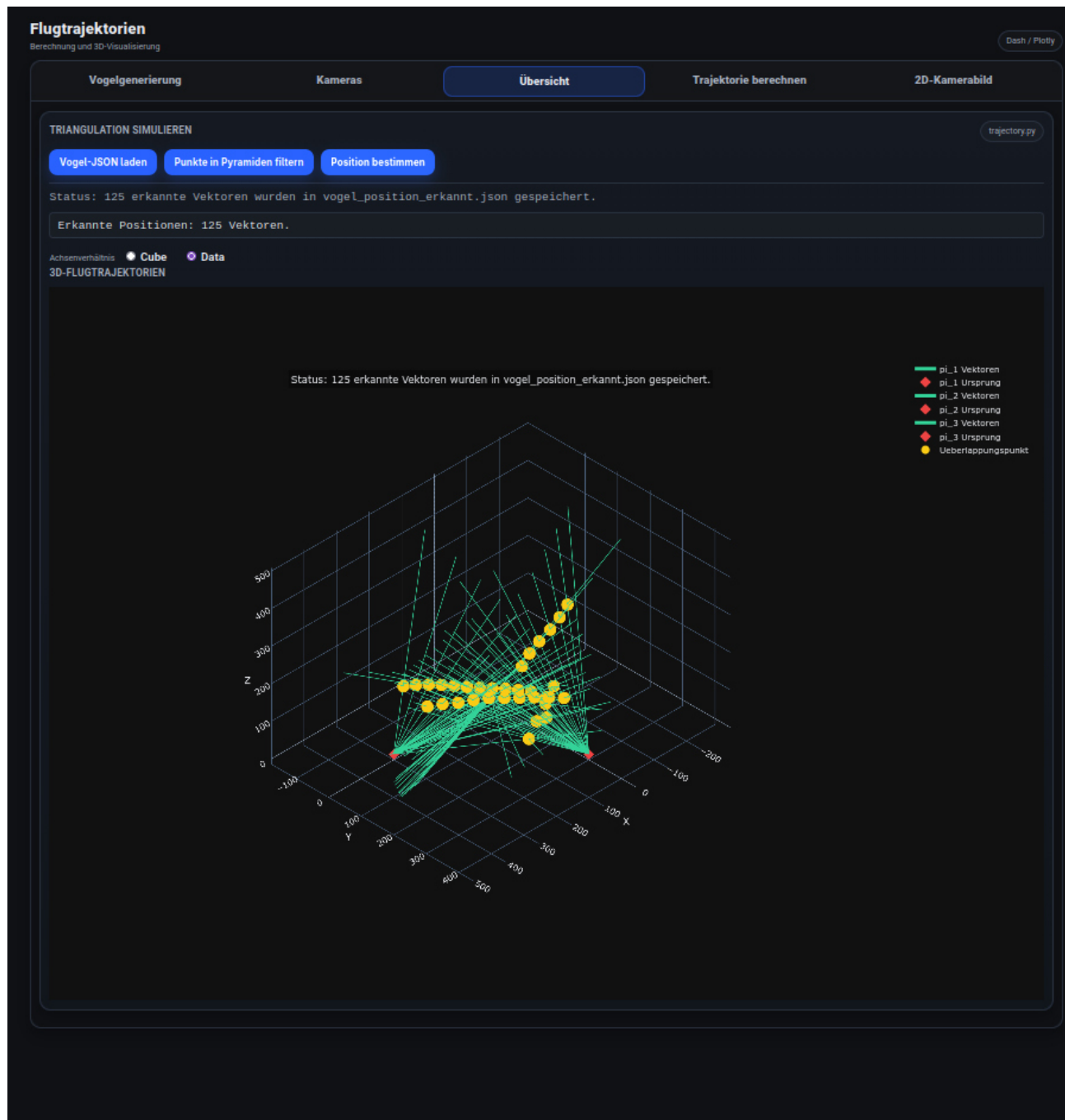


Abb. 8: Nach „Position bestimmen“ — grüne Sichtvektoren von allen drei Kameras, gelbe Überlappungspunkte = 125 triangulierte 3D-Positionen, gespeichert in `vogel_position_erkannt.json`.

4.4 Tab „Trajektorie berechnen“ (inkl. Feldlerchen-Klassifikation)

Dieser Tab hat drei Bereiche:

A) Trajektorien berechnen

Button „Trajektorie berechnen“ führt trajectory.py als separaten Prozess aus, bildet Tracks per Greedy-Zuordnung (Zeitstempel + Position), speichert vogel_flugbahnen_determined.json mit determined_track_id je Beobachtung. Konstanten: MAX_GAP_SECONDS=10, MAX_SPEED_UNITS_PER_SEC=25, DISTANCE_MARGIN=20.

B) Sichtbarkeit der Tracks (RadioItems)

- „**Alle Tracks anzeigen**“ (Standard) — jeder Track ist sichtbar.
- „**Nur Feldlerchen anzeigen**“ — nur die als Feldlerche klassifizierten Tracks sind initial sichtbar; setzt voraus, dass zuvor „Feldlerchen klassifizieren“ ausgeführt wurde.
- „**Alle Tracks ausblenden**“ — alle Tracks starten ausgeblendet.

Wichtig: Diese Auswahl setzt nur die Anfangssichtbarkeit. Jeder Track bleibt immer in der Legende und kann per Klick individuell ein-/ausgeblendet werden.

C) Feldlerchen-Klassifikation

Button „Feldlerchen klassifizieren“ ruft classify_tracks() aus skylark_classifier.py auf und zeigt das Ergebnis in einer sortierbaren Tabelle (Track, Feldlerche?, Score, Pfad) sowie Recall/Precision/Accuracy gegen die Simulations-Ground-Truth in der Statuszeile.

Funktionsweise des Klassifikators (regelbasiert, kein ML-Modell):

- **FeatureExtractor:** Höhengewinn, max. Steig-/Sinkrate, Plateauanteil (Singflug-Indikator), Rückkehr-Verhältnis (Revierbindung vs. Streckenflug), maximaler Radius um den Positionsmittelpunkt.
- **Pfad „Vollständig“:** hartes Gate — Steigflug UND Sturzflug erforderlich (Steigrate ≥ 1.0 m/s, Sinkrate ≥ 1.0 m/s, Höhengewinn ≥ 15.0 m). Danach gewichteter Score (Höhengewinn 25%, Plateauanteil 20%, Rückkehr-Verhältnis 30%, Aktionsradius 25%), Schwelle Score ≥ 0.6 .
- **Pfad „Isolierter Singflug“** (Ersatzpfad ohne Höhenbeweis): erfordert kreisende Bewegung (Radius ≥ 3.0 m, Pfad ≥ 10.0 m), strengere Schwelle Score ≥ 0.75 .
- **Pfad „Unvollständig“:** keine kreisende Bewegung erkennbar -> automatisch „kein Treffer“.

Alle Schwellenwerte liegen zentral in DEFAULT_THRESHOLDS in skylark_classifier.py.

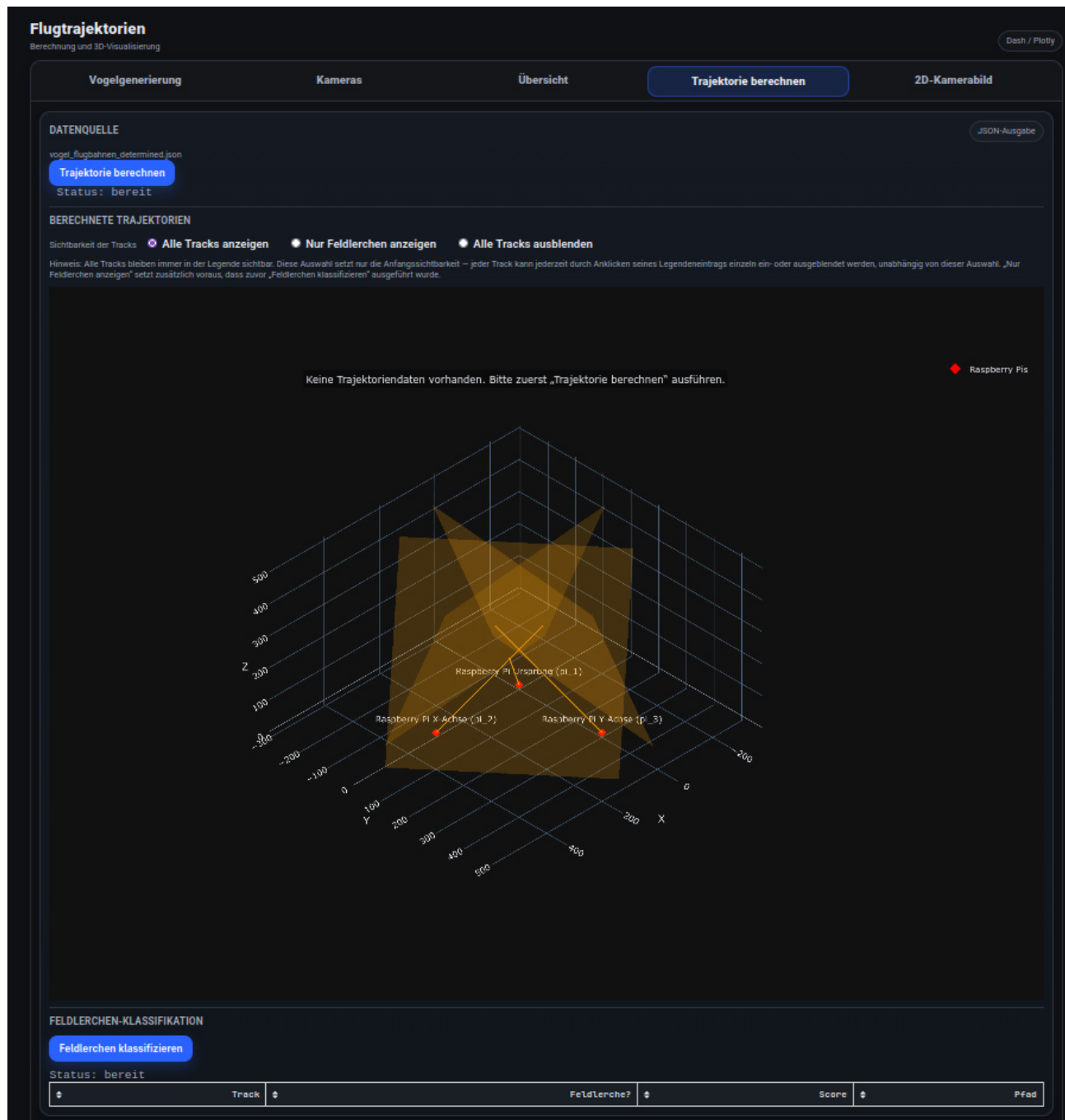


Abb. 9: Tab „Trajektorie berechnen“ im Ausgangszustand — noch keine Trajektorien berechnet, Klassifikationstabelle leer.

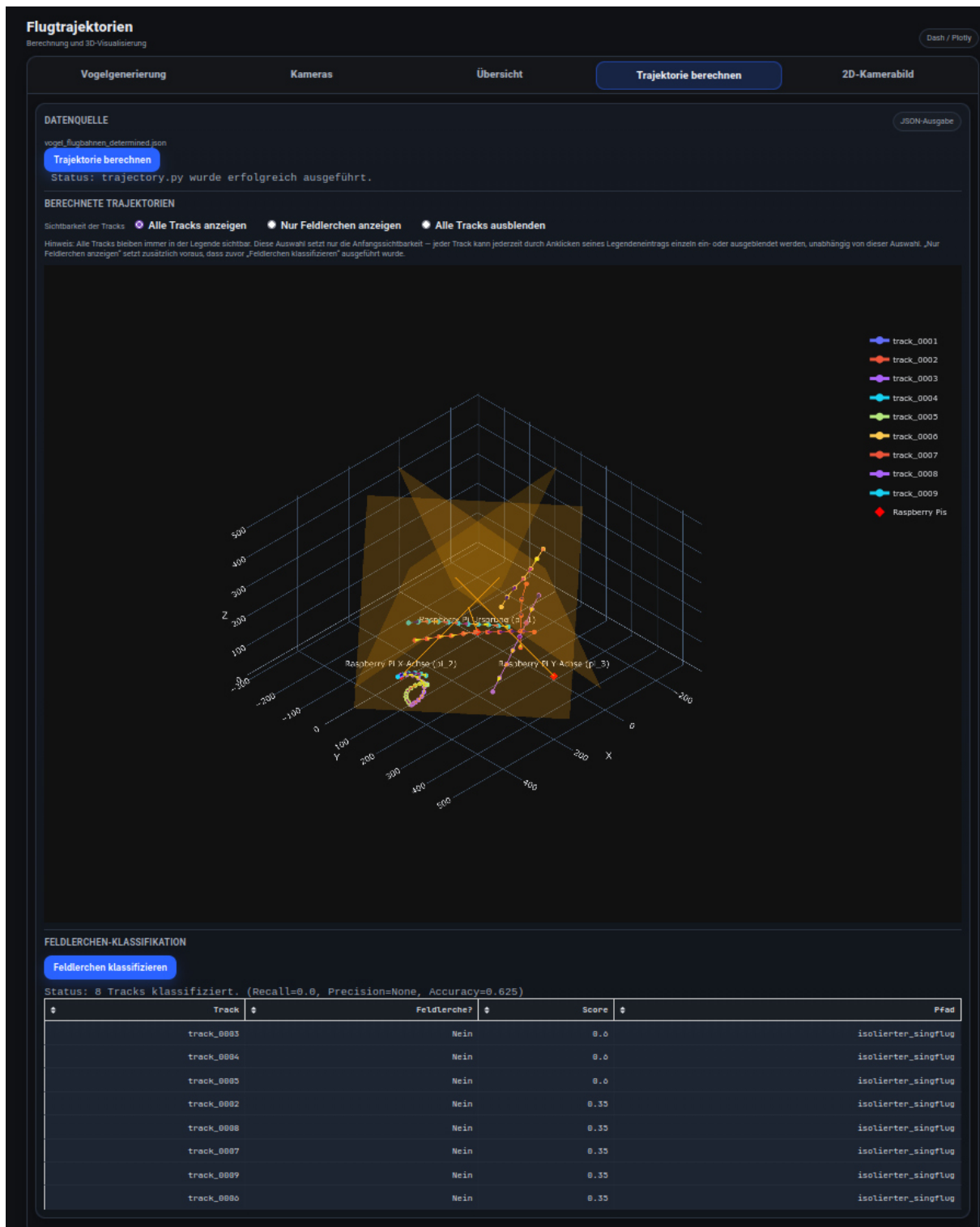


Abb. 10: Nach Berechnung und Klassifikation — 9 farblich unterschiedene Tracks im 3D-Graphen, Klassifikationstabelle mit Score und Pfad je Track. Statuszeile: Recall=0.0, Precision=None, Accuracy=0.625.

4.5 Tab „2D-Kamerabild“

Zeigt für jede der drei Kameras (pi_1, pi_2, pi_3) das simulierte Bild mit Bildzentrum (0,0) und den projizierten Vogelpunkten (orange), aus `vogel_bilder.json`. Voraussetzung: Der Filter-Schritt im Tab „Übersicht“ muss vorher gelaufen sein. Der Zeitfilter-Schieberegler erlaubt es, entweder alle Zeitpunkte gemeinsam oder nur einen einzelnen Zeitstempel anzuzeigen.

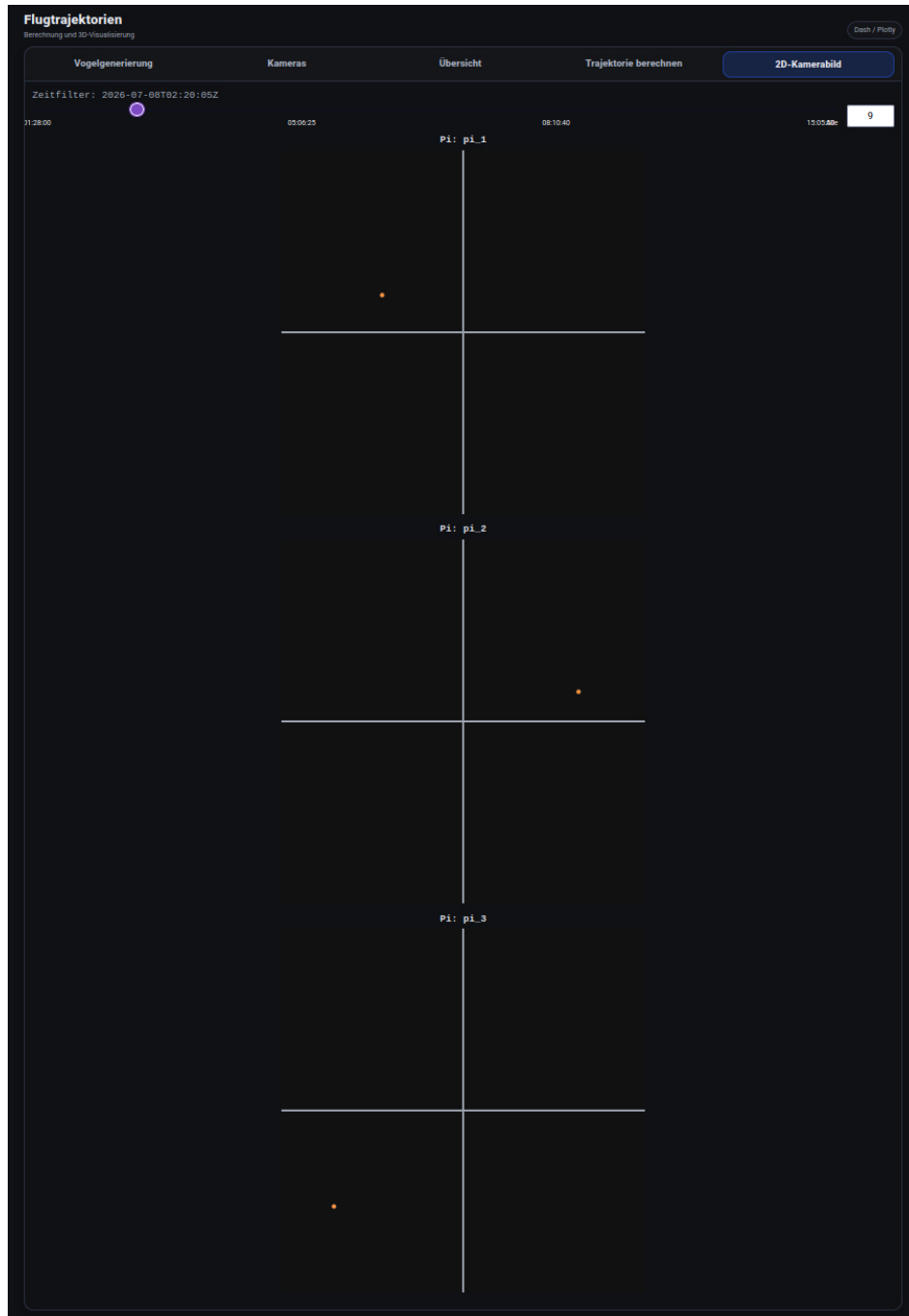


Abb. 11: 2D-Kamerabild aller drei Pis, gefiltert auf den Zeitpunkt 2026-07-08T02:20:05Z — je ein projizierter Vogelpunkt pro Kamera.

5. Datenfluss (Gesamtüberblick)

```

[Vogelgenerierung]
  generate_birds() + generate_skylarks() -> vogel_flugbahnen.json
    |
    v
[Übersicht: "Punkte in Pyramiden filtern"]
  points_in_view_volume_for_each_pi()
    |           |
    v           v
vogel_flugbahnen.json   vogel_bilder.json
    |
    v
[Übersicht: "Position bestimmen"] -> vogel_position_erkannt.json

[Trajektorie berechnen]
  trajectory.py -> vogel_flugbahnen_determined.json
    |
    v
[Feldlerchen klassifizieren] (Button im selben Tab)
  skylark_classifier.classify_tracks(records, group_column="determined_track_id")
  -> ClassificationResult pro Track (is_feldlerche, score, reasons, classification_path)
  -> Anzeige in Tabelle + lark-classification-store
  -> evaluate_against_ground_truth() -> Recall/Precision/Accuracy in Statuszeile
    |
    v
[Sichtbarkeitsfilter "Nur Feldlerchen anzeigen"]
  nutzt lark-classification-store fuer initiale Sichtbarkeit im 3D-Graphen

```

6. JSON-Datenformate

vogel_flugbahnen.json

```
{
  "plotAttachment": {"version": "1.0", "type": "triangulation_setup"},
  "simulated_birds": [
    {"bird_id": "bird_0001", "timestamp": "2026-07-08T12:00:00Z",
      "enu_e": 123.4, "enu_n": 56.7, "enu_u": 89.0},
    {"bird_id": "lark_0001", "determined_bird_id": "bird_0000",
      "timestamp": "2026-07-08T09:15:00Z",
      "enu_e": 210.0, "enu_n": 195.3, "enu_u": 78.2}
  ]
}
```

Feldlerchen sind über das lark_-Präfix in bird_id von normalen Vögeln unterscheidbar.

vogel_flugbahnen_determined.json

Gleiche Struktur wie oben, zusätzlich mit determined_track_id pro Beobachtung. Dies ist der Input für classify_tracks().

Klassifikations-Tabellenzeile (Anzeige, nicht persistiert)

```
{"track_id": "track_0007", "is_feldlerche": "Ja", "score": 0.812,
  "classification_path": "vollstaendig"}
```

Diese Ergebnisse werden aktuell **nicht** in eine eigene JSON-Datei geschrieben, sondern nur im Dash-Store lark-classification-store im Speicher gehalten (geht bei Neuladen der Seite verloren).

7. Typischer Arbeitsablauf (Schritt für Schritt)

- 1 Vogelgenerierung: Parameter setzen (inkl. Anzahl Feldlerchen) -> „JSON generieren“.
- 2 Kameras: Positionen/Winkel anpassen.
- 3 Übersicht: „Vogel-JSON laden“ -> „Punkte in Pyramiden filtern“ -> „Position bestimmen“.
- 4 2D-Kamerabild: Kontrollieren, was jede Kamera „gesehen“ hätte.
- 5 Trajektorie berechnen: Tracks aus den Beobachtungen bilden lassen.
- 6 Feldlerchen klassifizieren: Button klicken, Ergebnistabelle und Recall/Precision/Accuracy prüfen.
- 7 Sichtbarkeitsfilter nutzen: „Nur Feldlerchen anzeigen“ wählen, um erkannte Feldlerchen-Tracks hervorzuheben.

8. Bekannte Einschränkungen & offene Punkte

- **Klassifikationsergebnisse sind nicht persistent** — sie liegen nur im Dash-Store im Arbeitsspeicher und gehen bei einem Seiten-Reload verloren.
- **Keine UI zum Anpassen der Klassifikator-Schwellenwerte** — `DEFAULT_THRESHOLDS` ist nur im Code änderbar.
- Die Kamera-Sichtweite (`view_length`) ist hart auf 400.0 codiert.
- „Punkte in Pyramiden filtern“ überschreibt `vogel_flugbahnen.json` unwiderruflich.
- Keine Undo-Funktion, keine Versionierung der JSON-Dateien.
- `trajectory.py` läuft mit Timeout 300 s.
- Keine automatisierten Tests für die Dash-Callbacks oder den Klassifikator selbst (nur für Geometrie-Funktionen, siehe `test_visible_birds.py`).
- Keine Benutzer-Authentifizierung.

9. Wo man im Code weiterarbeitet

Ich möchte ...	... dann hier ansetzen
Klassifikationsergebnisse dauerhaft speichern	webinterface.py: <code>classify_larks()</code> -Callback erweitern, zusätzlich in JSON-Datei schreiben
Klassifikator-Schwellenwerte über UI einstellbar machen	webinterface.py: neue Input-Felder + Übergabe an <code>classify_tracks(..., thresholds=...)</code>
Klassifikator-Güte verbessern/tunen	skylark_classifier.py: <code>DEFAULT_THRESHOLDS</code> , <code>classify_features()</code>
Neue Flugmuster/Arten hinzufügen	bird_generation.py, nach Vorbild von skylark_generation.py
Kamera-Geometrie ändern	pi_view_simulation.py
Neue Visualisierungen/Plots	generate_plots.py
Track-Zuordnung verbessern	trajectory.py
Neue UI-Elemente	helper_ui.py + jeweiliges Tab-Layout

10. Kontakt & Übergabe-Hinweise

Bei Rückfragen zur Simulation zuerst dieses Dokument und die Kommentare im Code konsultieren. `skylark_classifier.py` ist ausführlich dokumentiert (Docstring am Dateianfang) — dort steht auch, dass die Kalibrierung gegen `skylark_generation.py` sowie gegen veröffentlichte Feldbeobachtungen (Singflughöhe 50-100 m, Drei-Phasen-Struktur, Revierbindung) erfolgt ist.

Die Callbacks in `webinterface.py`, die Klassifikation und Sichtbarkeitsfilter steuern, finden sich direkt nach den Trajektorie-Callbacks (`classify_larks`, `update_trajectory_visibility`).